

Algorithm Techniques

Dijkstra's Shortest Path & Heap Sort Algorithm

Group Assignment 2

Course: Algorithms Analysis and Design

Due Date: 28th May 2026

Presentation Overview



Dijkstra's Path

Exploring the greedy strategy for finding optimal routes in non-negative weighted graphs.



Heap Sort

Mastering the binary heap data structure to achieve consistent sorting performance.



Performance

Analyzing Big O complexities and real-world efficiency for management decisions.

Part 01

Dijkstra's Algorithm

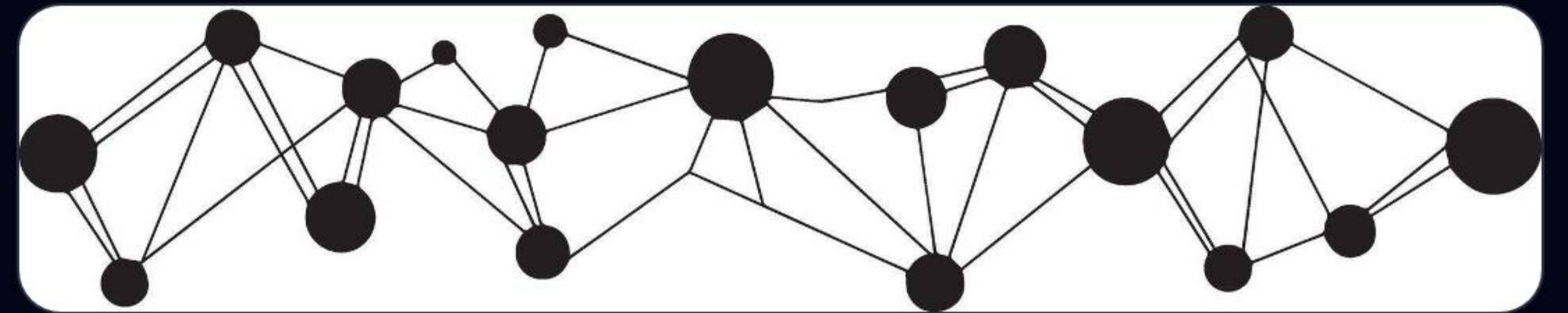
Shortest Path Discovery in Graph Theory

| Introduction to Dijkstra

The Greedy Approach

Dijkstra's algorithm is a single-source shortest path algorithm designed for graphs with non-negative edge weights.

- ✓ **Objective:** Finds the shortest path to all nodes.
- ✓ **Strategy:** Always picks the nearest unvisited node.
- ✓ **Constraint:** Edge weights must be non-negative.



| How it Works: The Logic

1

Initialize

Set Source distance to 0 and all others to ∞ .
Create unvisited set.

2

Select

Pick the unvisited node with the minimum current distance
(Greedy choice).

3

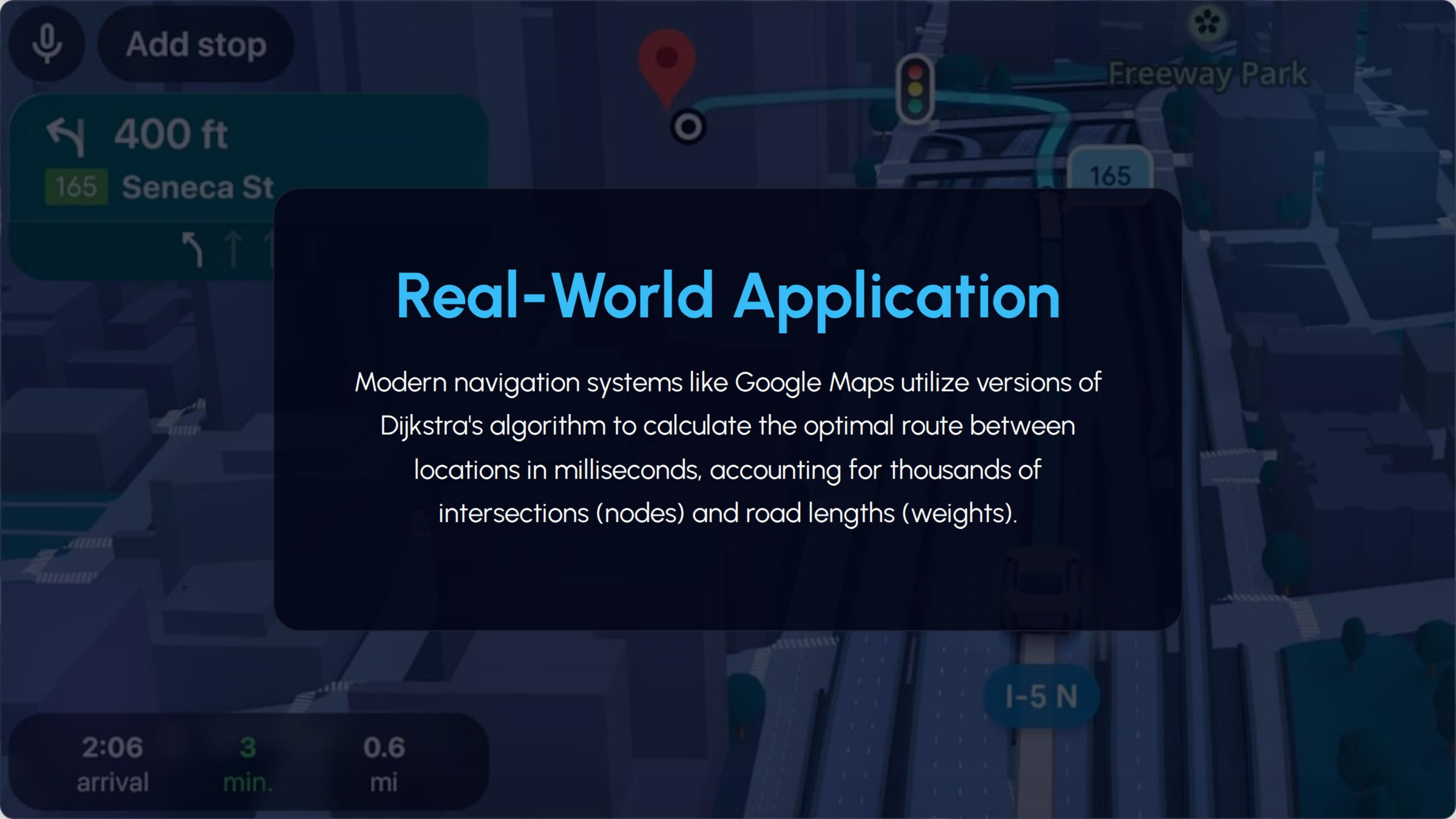
Relaxation

For each neighbor, if path through current is shorter than recorded, update distance.

4

Finalize

Repeat until all nodes are visited. Shortest paths are now determined.



Real-World Application

Modern navigation systems like Google Maps utilize versions of Dijkstra's algorithm to calculate the optimal route between locations in milliseconds, accounting for thousands of intersections (nodes) and road lengths (weights).

2:06
arrival

3
min.

0.6
mi

| Dijkstra Performance Analysis

$$O((V+E) \log V)$$

TIME COMPLEXITY

Efficiency Breakdown

When implemented with a *Min-Priority Queue* (Binary Heap):

$$T(n) = O((V + E) \log_2 V)$$

Space Complexity: $O(V)$ to maintain the distance table and priority queue.

Part 02

Heap Sort Algorithm

In-Place Sorting via Binary Heaps

| Understanding Heap Sort

Binary Heap Data Structure

Heap sort is a comparison-based algorithm that organizes elements into a **Max-Heap**: a complete binary tree where the parent is always greater than its children.

🔧 **Efficiency:** Guaranteed performance in worst-case.

🏠 **Space:** Zero auxiliary space required (In-place).

🔗 **Type:** Non-stable sorting.

 Binary Heap Diagram

| How It Works: Step-by-Step

Build Heap

Rearrange the array into a Max-Heap structure starting from bottom-most

pc ● ts.

Extract Max

Swap root (largest) with last element.
Reduce heap size by one.



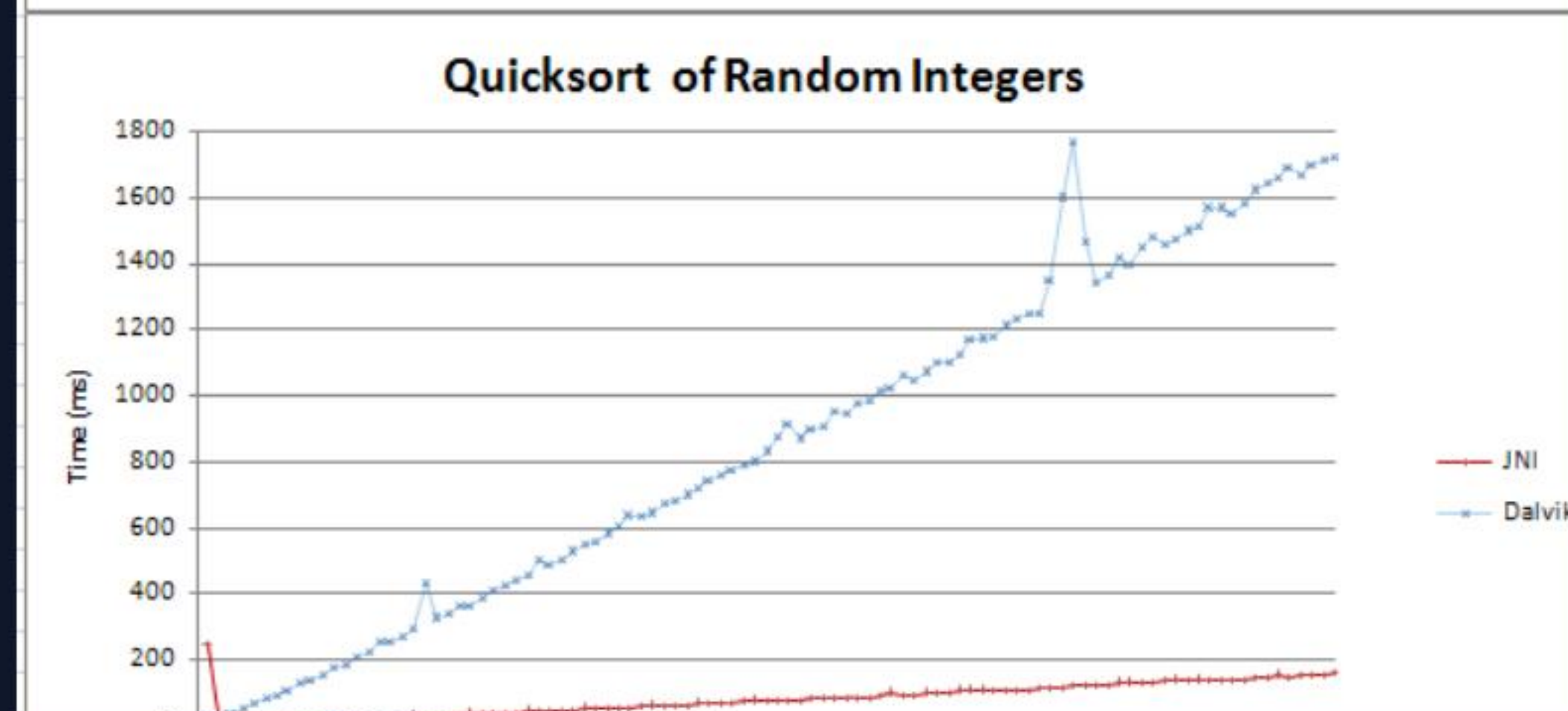
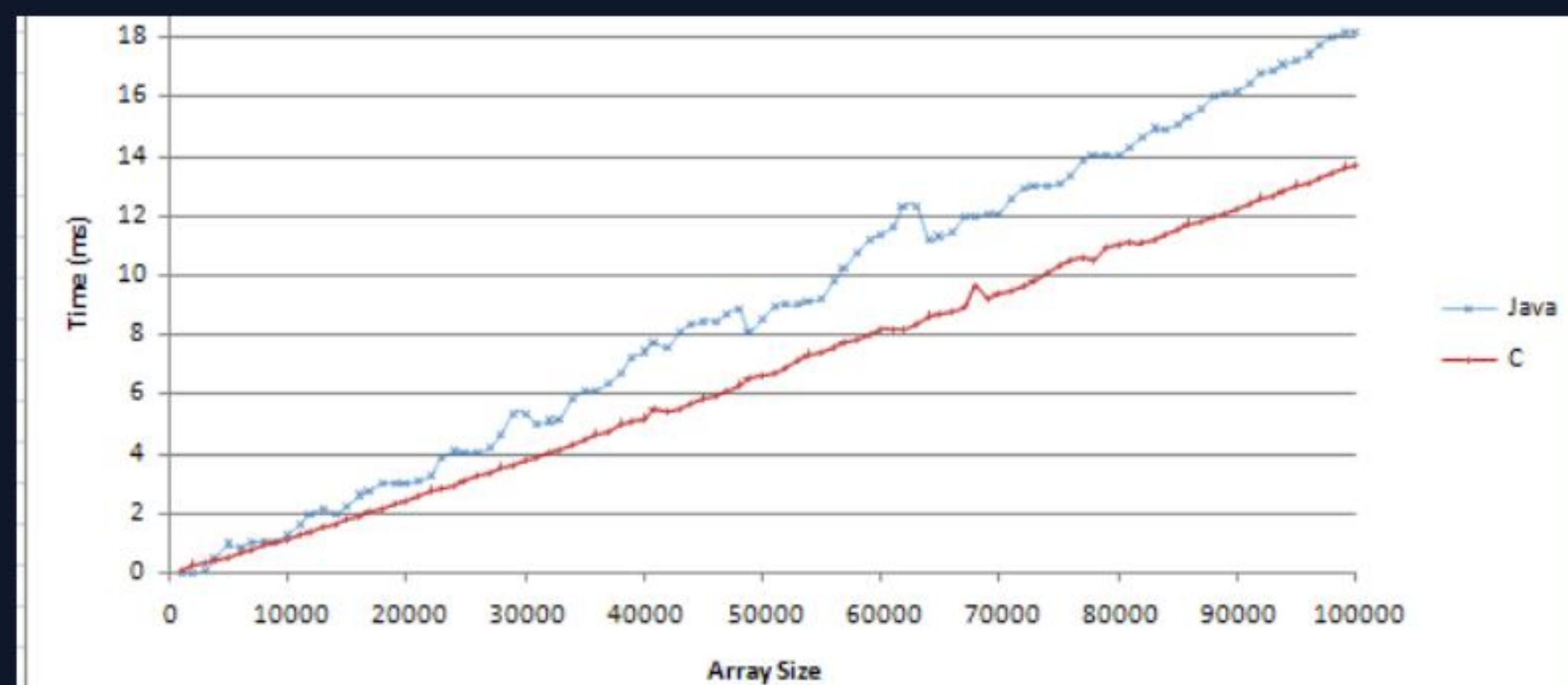
Re-Heapify

Run 'Heapify' on root to restore Max-Heap property. Repeat until sorted.



Example Trace: [2, 7, 1, 13, 20, 9, 7] → Max-Heap [20, 13, 9, 7, 2, 1, 7]

Heap Sort Analysis



Performance Stability

$$T(n) = O(n \log_2 n)$$

Complexity Highlights:

- **Worst Case:** $O(n \log n)$ - Very predictable.
- **Best Case:** $O(n \log n)$ - Consistent across inputs.
- **Space:** $O(1)$ - Superior memory efficiency.

Q&A

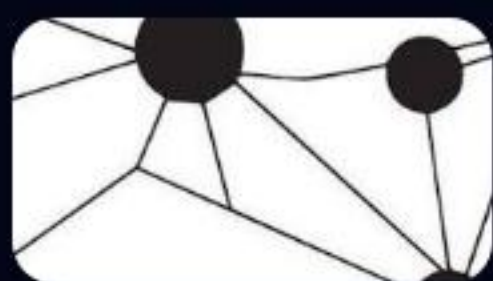
Thank You for Your Attention!

Prepared by Group 2 | ALGORITHMS ANALYSIS AND DESIGN

 assignment@university.edu

 algorithm-hub.edu

| Image Sources



https://static.vecteezy.com/system/resources/previews/067/936/241/non_2x/abstract-network-connecting-nodes-visual-representation-ideal-for-tech-science-data-visualization-vector.jpg

Source: www.vecteezy.com



Thumbnail for

<https://ik.imagekit.io/upgrad1/abroad-images/imageCompo/images/image4QX4V9J.png?pr=true>

www.upgrad.com Source: www.upgrad.com



https://lh4.ggpht.com/_EW60jqE5_B0/SiwLQa3-rYI/AAAAAAAAAEA/o0-NsWLrIxc/s800/Quicksort.png

Source: stackoverflow.com



Thumbnail

<https://cdn.shopify.com/s/files/1/0809/1975/7122/files/Google-Maps-Immersive-Navigation-3.jpg?v=1773717734>

for

Source: evdances.com

evdances.com